Principles in Practice - Draft a one-page scenario where you apply Microservices Architecture and Event-Driven Architecture to a hypothetical e-commerce platform. Outline how SOLID principles could enhance the design. Use bullet points to indicate how DRY and KISS principles can be observed in this context.

ShopSphere is an online marketplace designed using **Microservices Architecture** to promote scalability and modularity. Each core business capability—like orders, payments, inventory, and shipping—is developed as an independent microservice. These services communicate asynchronously using **Event-Driven Architecture**, where events such as OrderPlaced, PaymentSuccessful, or InventoryLow are published and consumed via a message broker (e.g., Kafka or RabbitMQ).

Application of Microservices & Event-Driven Architecture

- **Order Service** publishes an OrderPlaced event after validating a customer's purchase.
- **Payment Service** listens for OrderPlaced, initiates payment, and emits PaymentSuccessful or PaymentFailed.
- **Inventory Service** listens for PaymentSuccessful and reduces stock levels, triggering InventoryLow if thresholds are crossed.
- **Shipping Service** starts the delivery process after receiving PaymentSuccessful and emits OrderShipped.

Each service is independently deployable, fault-tolerant, and scalable.

Applying SOLID Principles

- **Single Responsibility Principle (SRP):**
    - Each service handles one distinct domain concern (e.g., payment processing, shipping logistics).
- **Open/Closed Principle (OCP):**
    - Services can extend functionality via new event listeners or emitters without altering existing code. E.g., adding a LoyaltyService that listens to PaymentSuccessful.
- **Liskov Substitution Principle (LSP):**
    - Interfaces or contracts (like IPaymentProcessor) ensure components like Stripe or PayPal integrations are interchangeable without side effects.

Observing DRY and KISS Principles

- **DRY (Don't Repeat Yourself):**
    - Common logic (e.g., input validation, logging) is abstracted into shared libraries or middleware.
    - Event schemas are standardized and reused across services.
- **KISS (Keep It Simple, Stupid):**
    - Services follow a clear event lifecycle, avoiding complex orchestration.
    - Message formats and error handling strategies are uniform to reduce cognitive load.